Exno: 12
Date 17.9.26

a) ECHO CLIENT-SERVER USING TCP/UDP SOCKETS

Aim: To implement an echo client-server application using TCP and UDP Sockets.

Procedure:

1. Create and compile the TCP and UDP server/client programs.
2. Start the appropriate server in one terminal.
3. Start the client in another terminal and enter a test message.
4. Verify that the server returns the received message with an echo prefix.

TCP Echo server - TCPEchoServer.java

```
import java.io.*;
import java.net.*;
public class TCPEchoServer {
    public static void main(String[] args) throws IOException {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo server listening on port " + port);
            while (true) {
                try (Socket clientSocket = serverSocket.accept()) {
                    System.out.println("client connected: " + clientSocket.getRemoteSocketAddress());
                    BufferedReader in = new BufferedReader(new InputStreamReader
                                        (clientSocket.getInputStream()));
                    PrintWriter out = new BufferedReader(new InputStreamReader
                                        (clientSocket.getOutputStream(), true));
                    String line;
                    while ((line = in.readLine()) != null) {
                        System.out.println("Received: " + line);
                        out.println("Echo: " + line);
                        if (line.equalsIgnoreCase("bye")) break;
                    }
                    System.out.println("client disconnected");
                } catch (IOException e) {
                    System.out.println("connection error: " + e.getMessage());
                }
            }
        }
    }
}
```

TCP Echo client - TCPEchoClient.java:

```
import java.io.*;
import java.net.*;
public class TCPEchoClient {
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;
        try (Socket socket = new Socket(host, port)) {
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            BufferedReader console = new BufferedReader(new InputStreamReader(System.in));
```

```
System.out.println("TCP Echo server at " + host + ":" + port);
System.out.println("Type message (type 'bye' to quit):");
String message;
while ((message = console.readLine()) != null) {
    out.println(message);
    String reply = in.readLine();
    System.out.println("Server: " + reply);
    if (message.equalsIgnoreCase("bye")) break;
}
}
System.out.println("Connection closed.");
}
}
```

UDP Echo Server – UDPEchoServer.java:

```
import java.net.*;
import java.nio.charset.StandardCharsets;
public class UDPEchoServer {
    public static void main(String[] args) throws Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
        byte[] buffer = new byte[1024];
        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Echo server listening on port " + port);
            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);
                String message = new String(request.getData(), 0, request.getLength(),
                                            StandardCharsets.UTF_8);
                System.out.println("Received: " + message);
                byte[] responseData = ("Echo: " + message).getBytes(StandardCharsets.UTF_8);
                DatagramPacket response = new DatagramPacket(responseData,
                        responseData.length, request.getAddress(), request.getPort());
                socket.send(response);
            }
        }
    }
}
```

UDP Echo client – UDPEchoClient.java:

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
public class UDPEchoClient {
    public static void main(String[] args) throws Exception {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 6000;
        InetAddress address = InetAddress.getByName(host);
        try (DatagramSocket socket = new DatagramSocket();
             BufferedReader userIn = new BufferedReader(new InputStreamReader(System.in))) {
            socket.setSoTimeout(5000);
            System.out.println("UDP Echo client -> " + host + ":" + port);
            System.out.println("Type message (type 'bye' to quit):");
            String message;
```

```
while(message = console.readLine() != null) {
    byte[] data = message.getBytes(StandardCharsets.UTF_8);
    Socket.send(new DatagramPacket(data, data.length, address, port));
    byte[] buffer = new byte[1024];
    DatagramPacket reply = new DatagramPacket(buffer, buffer.length);
    try {
        Socket.receive(reply);
        System.out.println("Server: " + new String(reply.getData(), 0, reply.getLength(),
                                                    StandardCharsets.UTF_8);
    } catch (SocketTimeoutException e) {
        System.out.println("No response received (timeout).");
    }
    if(message.equalsIgnoreCase("bye")) break;
}
}
}
```

Result:

Echo client-server communication was implemented and tested using TCP and UDP Sockets.

5) CHAT PROGRAM USING SOCKET PROGRAMMING

Aim:

To implement a client-server chat program using socket programming.

Procedure:

1. Compile the chat server and client.
2. Start the server, then launch two or more clients in separate terminals.
3. Enter client names and exchange messages.
4. Enter /quit to disconnect a client.

Chat Server.java

```
import java.io.*;
import java.net.*;
import java.util.*;
public class ChatServer {
    private static final int PORT = 5000;
    private static final Set<ClientHandler> clients = Collections.synchronizedSet(new
                                                      HashSet<>());

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(PORT)) {
            System.out.println("Chat server started on port " + PORT);
            while(true) {
                Socket socket = server.accept();
                ClientHandler handler
                clients.add(handler);  = new ClientHandler(socket);
                new Thread(handler).start();
            }
        }
    }

    private static void broadcast(String message, ClientHandler sender) {
        synchronized (clients) {
            for (ClientHandler client : clients) {
                if(client != sender) client.send(message);
            }
        }
    }
    private static class ClientHandler implements Runnable {
        private final Socket socket;
        private BufferedReader in;
        private PrintWriter out;
        private String name = "Anonymous";
        ClientHandler(Socket socket) { this.socket = socket; }
        void send(String message) {
            if (out != null) out.println(message);
        }
        public void run() {
            try {
                in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
                out = new PrintWriter(socket.getOutputStream(), true);
                out.println("Enter your name: ");
                String suppliedName = in.readLine();
                if(suppliedName != null && !suppliedName.isBlank()) name = suppliedName;
```

```
broadcast(name + " has joined the chat", this);
out.println("Welcome, " + name + "! Type /quit to exit.");
String line;
while ((line = in.readLine()) != null) {
    if (line.equalsIgnoreCase("/quit")) break;
    System.out.println(name + ": " + line);
    broadcast(name + ": " + line, this);
}
} catch (IOException e) {
    System.out.println("Client error: " + e.getMessage());
} finally {
    clients.remove(this);
    broadcast(name + " has left the chat.", this);
    try { socket.close(); } catch (IOException e) {}
}
}
}
```

ChatClient.java

```
import java.io.*;
import java.net.*;
public class ChatClient {
    private static final String SERVER_ADDRESS = "127.0.0.1";
    private static final int SERVER_PORT = 5000;
    public static void main(String[] args) throws IOException {
        try (Socket socket = new Socket(SERVER_ADDRESS, SERVER_PORT)) {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader console = new BufferedReader(
                new InputStreamReader(System.in));
            Thread listener = new Thread(() -> {
                try {
                    String message;
                    while ((message = in.readLine()) != null) System.out.println(
                        message);
                } catch (IOException e) {
                    System.out.println("Disconnected from server.");
                }
            });
            listener.setDaemon(true);
            listener.start();
            String input;
            while ((input = console.readLine()) != null) {
                out.println(input);
                if (input.equalsIgnoreCase("/quit")) break;
            }
        }
    }
}
```

Result:

Output:

TCP Echo client-Server

Message : Hello

Echo : Hello

UDP Echo client-server

Message : UDP

Echo : UDP

Chat program - Server side

You : Hi

Server : Hello

You : How are you ?

Server : I am healthy

You : exit / quit

Chat program - client side

Client : Hi

You : Hello

Client : How are you?

You : I am healthy